

Week 15 Assessment: Northwind Catalog (DB First + API + MVC)

1. INTRODUCTION

This project is a 3-layer full-stack application developed using .NET technologies. It uses the Northwind database and implements the Database First approach using Entity Framework Core. The system is designed to display product categories and their respective products using a modern user interface. It also includes analytics features such as category-wise summaries.

The project follows a clean architecture with separation of concerns using Repository Pattern, DTOs, AutoMapper, and API-based communication between layers.

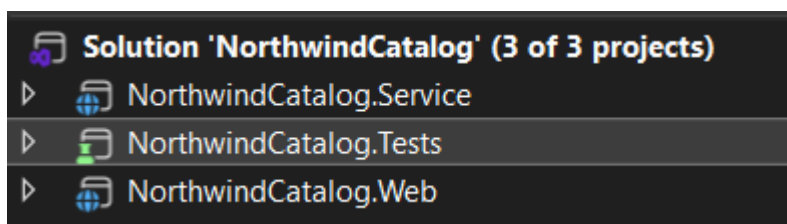
2. OBJECTIVES

- Implement DB First approach using EF Core
- Build RESTful APIs
- Consume APIs in MVC
- Design responsive UI
- Calculate Inventory Value
- Implement analytics
- Perform unit testing

3. PROJECT STRUCTURE

The solution consists of three projects:

1. Northwind.Services – API, EF Core, Repository
2. Northwind.Web – MVC UI
3. Northwind.Tests – Unit Testing



4. DATABASE FIRST APPROACH

The database models were generated using Scaffold-DbContext command in EF Core.

Tables used:

- Categories
- Products

Command: `dotnet ef dbcontext scaffold`

`"Server=.\\SQLEXPRESS;Database=Northwind;Trusted_Connection=True;TrustServerCertificate=True;" Microsoft.EntityFrameworkCore.SqlServer --output-dir Models --context-dir Data --context NorthwindContext --table Categories --table Products --force`

```
PS C:\Users\gurno\source\repos\NorthwindCatalog\NorthwindCatalog> dotnet ef dbcontext scaffold "Server=.\sqlexpress;Database=Northwind;Trusted_Connection=True;TrustServerCertificate=True;" Microsoft.EntityFrameworkCore.SqlServer --output-dir Models --context-dir Data --context NorthwindContext --table Categories --table Products --force
Build started...
Build succeeded.
```

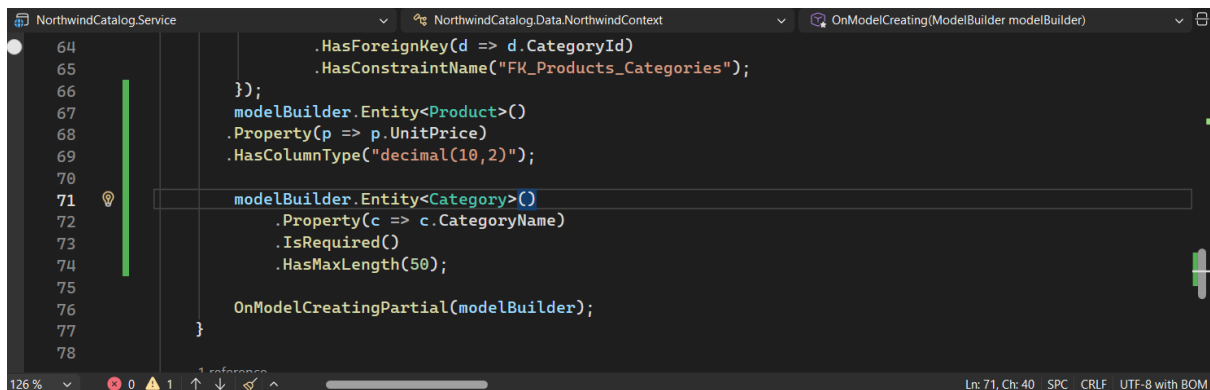
5. FLUENT API CONFIGURATION

Fluent API is used to configure entity properties and relationships.

Code:

```
modelBuilder.Entity<Product>()
    .Property(p => p.UnitPrice)
    .HasColumnType("decimal(10,2)");

modelBuilder.Entity<Product>()
    .HasOne(p => p.Category)
    .WithMany(c => c.Products)
    .HasForeignKey(p => p.CategoryId);
```



6. DTO IMPLEMENTATION

DTOs are used to transfer only required data between layers.

Code:

CategoryDto

```
public class CategoryDto
{
    public int CategoryId { get; set; }
    public string CategoryName { get; set; }
    public string ImageUrl { get; set; }
}
```

ProductDto

```
public class ProductDto
{
    public string ProductName { get; set; }
    public decimal UnitPrice { get; set; }
```

```

    public short UnitsInStock { get; set; }

    public decimal InventoryValue => UnitPrice * UnitsInStock;
}

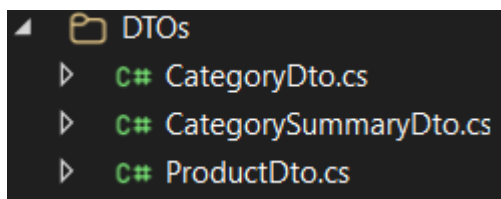
```

CategorySummaryDto

```

public class CategorySummaryDto
{
    public string CategoryName { get; set; }
    public int ProductCount { get; set; }
    public decimal AvgPrice { get; set; }
    public string MostExpensiveProduct { get; set; }
}

```



7. AUTOMAPPER

AutoMapper is used to map Entity Models to DTOs automatically.

Code:

```

public class MappingProfile: Profile
{
    public MappingProfile()
    {
        CreateMap<Category, CategoryDto>()
            .ForMember(dest => dest.ImageUrl,
                opt => opt.MapFrom(src => "/images/" + src.CategoryId + ".jpeg"));

        CreateMap<Product, ProductDto>();
    }
}

```

8. REPOSITORY PATTERN

Repository Pattern separates business logic from data access.

Code:

ICategoryRepository.cs

```

public interface ICategoryRepository
{
    Task<IEnumerable<Category>> GetAllAsync();
}

```

IPProductRepository.cs

```
public interface IPProductRepository
{
    Task<IEnumerable<Product>> GetByCategoryIdAsync(int categoryId);
    Task<IEnumerable<CategorySummaryDto>> GetCategorySummariesAsync();
}
```

Implementations

CategoryRepository.cs

```
using Microsoft.EntityFrameworkCore;

public class CategoryRepository : ICategoryRepository
{
    private readonly NorthwindContext _context;

    public CategoryRepository(NorthwindContext context)
    {
        _context = context;
    }

    public async Task<IEnumerable<Category>> GetAllAsync()
    {
        return await _context.Categories.ToListAsync();
    }
}
```

ProductRepository.cs

```
using Microsoft.EntityFrameworkCore;

public class ProductRepository : IPProductRepository
{
    private readonly NorthwindContext _context;

    public ProductRepository(NorthwindContext context)
    {
        _context = context;
    }

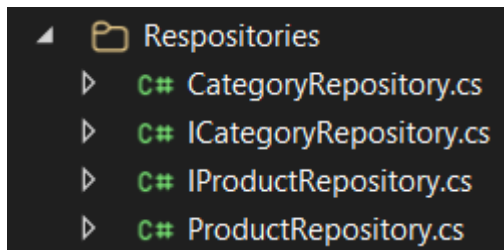
    public async Task<IEnumerable<Product>> GetByCategoryIdAsync(int categoryId)
    {
        return await _context.Products
```

```

        .Where(p => p.CategoryId == categoryId)
        .ToListAsync();
    }

    public async Task<IEnumerable<CategorySummaryDto>>
    GetCategorySummariesAsync()
    {
        return await _context.Categories
            .Select(c => new CategorySummaryDto
            {
                CategoryName = c.CategoryName,
                ProductCount = c.Products.Count(),
                AvgPrice = c.Products.Average(p => p.UnitPrice ?? 0),
                MostExpensiveProduct = c.Products
                    .OrderByDescending(p => p.UnitPrice)
                    .Select(p => p.ProductName)
                    .FirstOrDefault()
            }).ToListAsync();
    }
}

```



9. API IMPLEMENTATION

APIs were developed to expose data for categories, products, and summary.

CategoriesApiController.cs

```

[ApiController]
[Route("api/categories")]
public class CategoriesApiController : ControllerBase
{
    private readonly ICategoryRepository _repo;
    private readonly IMapper _mapper;

    public CategoriesApiController(ICategoryRepository repo, IMapper mapper)
    {
        _repo = repo;
        _mapper = mapper;
    }
}

```

```
[HttpGet]
public async Task<IActionResult> Get()
{
    var categories = await _repo.GetAllAsync();
    return Ok(_mapper.Map<IEnumerable<CategoryDto>>(categories));
}
}
```

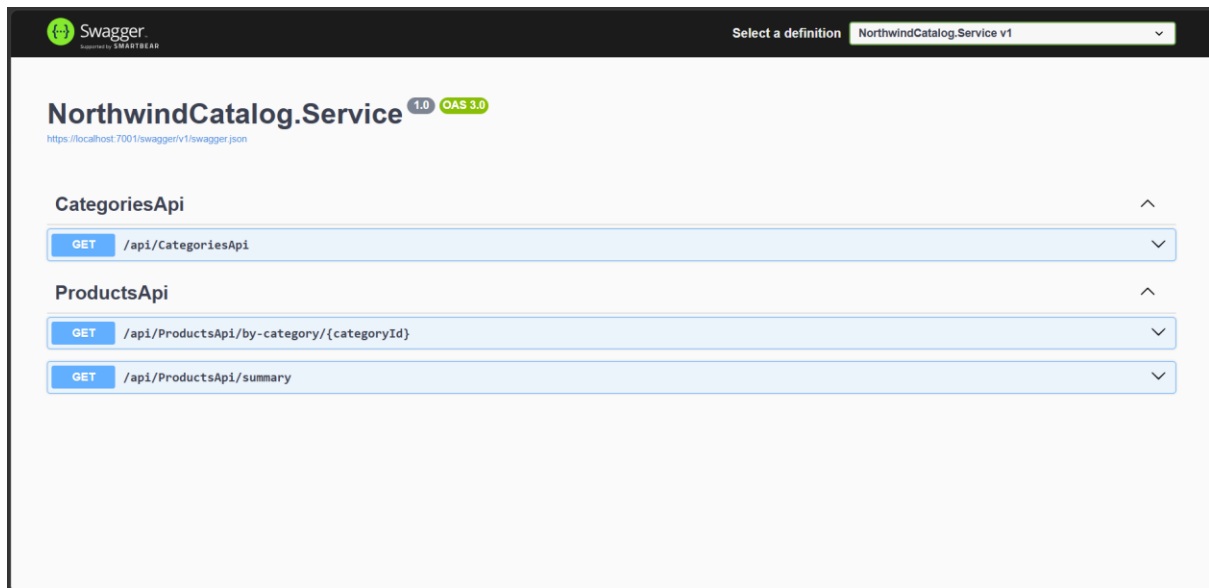
ProductsApiController.cs

```
[ApiController]
[Route("api/products")]
public class ProductsApiController : ControllerBase
{
    private readonly IProductRepository _repo;
    private readonly IMapper _mapper;

    public ProductsApiController(IProductRepository repo, IMapper mapper)
    {
        _repo = repo;
        _mapper = mapper;
    }

    [HttpGet("by-category/{categoryId}")]
    public async Task<IActionResult> GetByCategory(int categoryId)
    {
        var products = await _repo.GetByCategoryIdAsync(categoryId);
        return Ok(_mapper.Map<IEnumerable<ProductDto>>(products));
    }

    [HttpGet("summary")]
    public async Task<IActionResult> GetSummary()
    {
        var data = await _repo.GetCategorySummariesAsync();
        return Ok(data);
    }
}
```



10. MVC Controller

Code:

CategoriesController

```
Public class CategoriesController : Controller
{
    private readonly HttpClient _client;

    public CategoriesController(IHttpClientFactory factory)
    {
        _client = factory.CreateClient("api");
    }

    public async Task<IActionResult> Index()
    {
        var data = await
        _client.GetFromJsonAsync<List<CategoryDto>>("api/categoriesApi");
        return View(data);
    }
}
```

ProductsController:

```
public class ProductsController : Controller
{
    private readonly HttpClient _client;

    public ProductsController(IHttpClientFactory factory)
    {
        _client = factory.CreateClient("api");
    }
}
```

```

public async Task<IActionResult> ByCategory(int id)
{
    var products = await _client.GetFromJsonAsync<List<ProductDto>>
        ($"api/ProductsApi/by-category/{id}");

    return View(products);
}
}

```

SummaryController:

```

public class SummaryController : Controller
{
    private readonly HttpClient _client;

    public SummaryController(IHttpClientFactory factory)
    {
        _client = factory.CreateClient("api");
    }

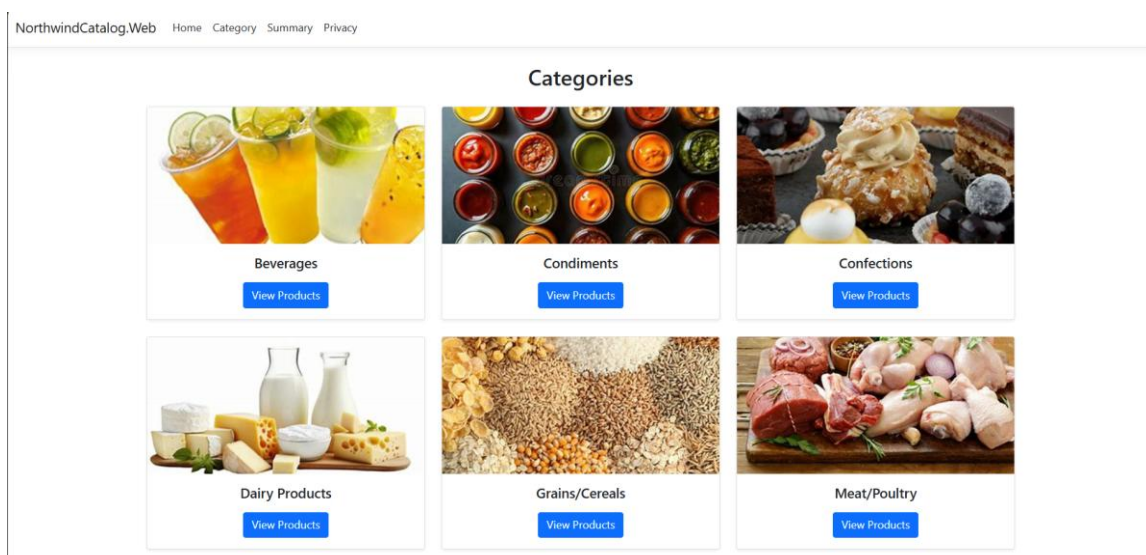
    public async Task<IActionResult> Index()
    {
        var data = await _client.GetFromJsonAsync<List<CategorySummaryDto>>
            ("api/ProductsApi/summary");

        return View(data);
    }
}

```

10. UI Implementation

Categories page:



Products page:

NorthwindCatalog.Web Home Category Summary Privacy

Products

Chai ₹ 18.0000 Stock: 39 ₹ 702.0000 Inventory Value	Updated Chang ₹ 19.0000 Stock: 17 ₹ 323.0000 Inventory Value	Guaraná Fantástica ₹ 4.5000 Stock: 29 ₹ 90.0000 Inventory Value
Sasquatch Ale ₹ 14.0000 Stock: 111 ₹ 1554.0000 Inventory Value	Steeleye Stout ₹ 18.0000 Stock: 20 ₹ 360.0000 Inventory Value	Côte de Blaye ₹ 263.5000 Stock: 17 ₹ 4479.5000 Inventory Value
Chartreuse verte ₹ 18.0000 Stock: 69 ₹ 1242.0000 Inventory Value	Ipoh Coffee ₹ 46.0000 Stock: 17 ₹ 782.0000 Inventory Value	Laughing Lumberjack Lager ₹ 14.0000 Stock: 52 ₹ 728.0000 Inventory Value

Summary page:

NorthwindCatalog.Web Home Category Summary Privacy

Category Insights

Total: 8

Category	Products	Avg Price	Top Product
Beverages	12	₹ 37.9791	Côte de Blaye
Condiments	12	₹ 23.0625	Veggie-spread
Confections	13	₹ 25.1600	Sir Rodney's Marmalade
Dairy Products	10	₹ 28.7300	Raclette Courdavault
Grains/Cereals	7	₹ 20.2500	Gnocchi di nonna Alice
Meat/Poultry	6	₹ 54.0066	Thüringer Rostbratwurst
Produce	5	₹ 32.3700	Manjimup Dried Apples
Seafood	12	₹ 20.6825	Carnarvon Tigers

© 2026 - NorthwindCatalog.Web - [Privacy](#)

11. UNIT TESTING

Unit testing is done using xUnit.

Code:

```
public class ProductTests
{
    [Fact]
    public void InventoryValue_Should_Return_Correct_Value()
    {
        var product = new ProductDto
        {
            UnitPrice = 20,
```

```
        UnitsInStock = 5  
    };  
    Assert.Equal(100, product.InventoryValue);  
}  
}
```

